Day 17 Task Workflow — Monday, Mar 30, 2026

> **Sprint:** 5 (Feature Depth → Transition to Sprint 6) | **Phase:** CICIDS2017 Validation + v0.5.0 Finalization + Week 6 Kickoff
> **Owner:** Lead Architect | **Status:** ⚪ Ready to Start
> **Goal:** Complete CICIDS2017 validation (Day 16 carryover), finalize v0.5.0, begin Week 6 enterprise scaffolding
> **Grade:** Week 5 Day 1 (Day 16) COMPLETE ☑ | Week 5 Day 2 STARTING ⚫

Day 16 Results Context

14/14 verification checks passed (100%) across Tasks 1, 3, 4, 5:

```
Day 16 Achievements:
  ☑ PCAP Processor: 556 lines, integrated with upload endpoint
  ☑ ml_models Table: 3 entries (IF v1.1, RF v1.0, AE v1.0), metrics populated
  ☑ ML Ops Endpoints: confusion-matrix, feature-importance, training-history
  ☑ Admin Audit Log: GET /admin/audit-log scaffold operational
  ☑ Bug Fixed: populate_ml_models.py DuplicateTableError (asyncpg driver-level)
  ⏳ CICIDS2017 Validation: Deferred (loader ready, dataset download required)
  ☑ API Coverage: 46/46 (100%) 🎯 (+4 from Day 15)

Container Status:
  tm-backend    ☑ Up 3+ days (all new endpoints live)
  tm-capture    ☑ Up 3+ days
  tm-ml-worker  ☑ Up 3+ days (tuned IF active)
  tm-postgres   ☑ Healthy 3+ days (ml_models populated)
  tm-redis      ☑ Healthy 3+ days
```

Scope Adherence Checklist

Requirement	Source Document	Section	Status
CICIDS2017 validation (carryover)	MASTER_DOC_PART4	§3, PART5 Week 5	❌ **DO TODAY**
PDF report generation	MASTER_DOC_PART5	Week 6	⏳ Begin scaffolding
RBAC enforcement	MASTER_DOC_PART5	Week 6	⏳ Begin scaffolding
LLM budget tracking	MASTER_DOC_PART5	Week 6	⏳ Begin scaffolding
Audit log event wiring	MASTER_DOC_PART3	§11.1	⏳ Begin wiring
Ensemble weights (0.30/0.45/0.25)	MASTER_DOC_PART4	§1.2	🔒 LOCKED
Alert thresholds (0.90/0.75/0.50/0.30)	MASTER_DOC_PART4	§1.2	🔒 LOCKED

LOCKED CONSTRAINTS (DO NOT MODIFY)

```
Ensemble Weights:
  composite = 0.30 × IF + 0.45 × RF + 0.25 × AE

Alert Thresholds:
  ≥ 0.90 → CRITICAL
  ≥ 0.75 → HIGH
  ≥ 0.50 → MEDIUM
  ≥ 0.30 → LOW
  < 0.30 → NONE

LLM Provider: OpenRouter only (3 verified models)
DO NOT suggest: Kafka, Kubernetes, Elasticsearch, MongoDB
```

Day 17 Objective

By end of day:

- CICIDS2017 ensemble validation executed — results saved to eval JSON, accessible via `GET /ml/comparison`
- PDF report generation scaffolded (ReportLab integration)
- Audit log events wired into critical actions (login, retrain, alert status change)
- RBAC middleware scaffolded for role-based endpoint protection
- LLM budget tracking endpoint enhanced with actual token usage data

> **NOTE:** Tasks 2-5 are early Week 6 items — starting them today leverages the Day 16 momentum to stay ahead of schedule.

Task Breakdown

TASK 1 — CICIDS2017 Validation Run 🔴 (Day 16 Carryover)

Time Est: 60 min | **Priority:** 🔴 Critical — Academic credibility + v0.5.0 completion
Source: MASTER_DOC_PART4 §3, MASTER_DOC_PART5 Week 5
Carryover From: Day 16 Task 2

The `CICIDS2017Loader` class (437 lines) and validation script (`scripts/run_cicids_validation.py`) exist. The download helper (`scripts/download_cicids2017.sh`) is ready.

1.1 Download CICIDS2017 Dataset Subset on VPS

```
# On VPS — create directory and download subset:
mkdir -p /app/ml/saved_models/datasets/cicids2017/

# Option A: Use download helper script
```

```
bash /app/scripts/download_cicids2017.sh

# Option B: Manual download (Friday DDoS subset, ~170 MB)
# Using a pre-hosted mirror or kaggle CLI
```

> **FALLBACK:** If the full dataset cannot be downloaded (network/storage), create a synthetic CICIDS2017-style validation dataset using the loader's feature mapping.

1.2 Execute Validation

```
docker compose exec backend python scripts/run_cicids_validation.py
```

Or directly:

```
docker compose exec backend python -c "
from ml.datasets.cicids2017 import validate_ensemble_on_cicids2017
results = validate_ensemble_on_cicids2017()
import json
print(json.dumps(results, indent=2, default=str))
"
```

1.3 Save + Expose Results

Results should be saved to `ml/saved_models/eval_results/cicids2017_validation.json` and appear in `GET /ml/comparison` response.

Verification:

#	Check	Expected
1	`validate_ensemble_on_cicids2017()` completes	Returns results dict
2	`cicids2017_validation.json` saved	File exists with metrics
3	Accuracy reported	Numeric value (cross-dataset performance)
4	F1 score reported	Numeric value
5	`GET /ml/comparison` includes CICIDS2017	Listed in model comparison
6	Label distribution logged	Class breakdown visible in output

---

TASK 2 — PDF Report Generation (ReportLab) 🟤

Time Est: 75 min | **Priority:** 🟤 Critical — Week 6 deliverable
Source: MASTER_DOC_PART5 Week 6 ("PDF report generation — Daily threat summary PDF")

The reports module exists with 3 endpoints (generate, list, download) but outputs JSON only. Add PDF generation.

2.1 Install ReportLab

Ensure `reportlab` is in `backend/requirements.txt`.

2.2 Create `backend/app/services/report_generator.py`

The PDF generator should:

1. Accept report data (alerts, stats, model performance)
2. Generate a branded PDF with ThreatMatrix AI header
3. Include sections: Executive Summary, Alert Summary, Top Threats, ML Model Status, IOC Summary
4. Save PDF to `/app/reports/` directory
5. Return file path for download endpoint

```
"""
ThreatMatrix AI — PDF Report Generator

Per MASTER_DOC_PART5 Week 6:
  "PDF report generation (ReportLab) — Daily threat summary PDF"

Generates branded PDF reports with:
  - Executive summary with threat level
  - Alert breakdown by severity
  - Top threats and IOC matches
  - ML model performance summary
"""

from reportlab.lib import colors
from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import getSampleStyleSheet
from reportlab.platypus import SimpleDocTemplate, Paragraph, Table, TableStyle,
Spacer
from pathlib import Path
from datetime import datetime, timezone
from typing import Any, Dict

# ... implementation
```

2.3 Update `reports.py` Download Endpoint

Modify `GET /reports/{id}/download` to serve PDF files when `format=pdf`.

Verification:

#	Check	Expected
1	`reportlab` importable in container	No ImportError

#	Check	Expected
2	`POST /reports/generate` with `format=pdf`	Returns report_id
3	PDF file created in `/app/reports/`	File exists, >0 bytes
4	`GET /reports/{id}/download` serves PDF	Content-Type: application/pdf
5	PDF contains branded header	"ThreatMatrix AI" title
6	PDF contains alert summary table	Severity breakdown

TASK 3 — Audit Log Event Wiring 🟢

Time Est: 45 min | **Priority:** 🟢 Medium — Makes audit log useful
Source: MASTER_DOC_PART3 §11.1 (Administration Module — Audit Log)

The `audit_log` table exists and `GET /admin/audit-log` is live, but no events are being recorded. Wire critical actions to write audit entries.

3.1 Create `backend/app/services/audit_service.py`

```
"""
ThreatMatrix AI — Audit Service

Per MASTER_DOC_PART3 §11.1:
  Records all significant system events for compliance and forensics.

Events to track:
  - User login/logout
  - Alert status changes
  - Model retrain triggers
  - System config changes
  - Report generation
  - IOC sync operations
"""

from __future__ import annotations

import logging
from typing import Any, Dict, Optional
from uuid import uuid4
from datetime import datetime, timezone

from sqlalchemy import text
from app.database import async_session

logger = logging.getLogger(__name__)


async def log_audit_event(
    action: str,
```

```
    entity_type: str,
    entity_id: Optional[str] = None,
    user_id: Optional[str] = None,
    details: Optional[Dict[str, Any]] = None,
    ip_address: Optional[str] = None,
) -> None:
    """
    Record an audit log entry.

    Args:
        action: Action performed (e.g., "login", "retrain", "alert_status_change")
        entity_type: Type of entity affected (e.g., "user", "model", "alert")
        entity_id: ID of the affected entity
        user_id: ID of the user who performed the action
        details: Additional context as JSONB
        ip_address: Client IP address
    """
    try:
        async with async_session() as session:
            await session.execute(
                text("""
                    INSERT INTO audit_log (id, user_id, action, entity_type,
entity_id,
                                           details, ip_address, created_at)
                    VALUES (:id, :user_id, :action, :entity_type, :entity_id,
                            :details::jsonb, :ip_address::inet, :now)
                """),
                {
                    "id": str(uuid4()),
                    "user_id": user_id,
                    "action": action,
                    "entity_type": entity_type,
                    "entity_id": entity_id,
                    "details": json.dumps(details) if details else None,
                    "ip_address": ip_address,
                    "now": datetime.now(timezone.utc),
                },
            )
            await session.commit()
    except Exception as e:
        logger.error("[AUDIT] Failed to log event: %s — %s", action, e)
```

3.2 Wire Into Critical Endpoints

Add `log_audit_event()` calls to:

- `auth.py` — POST /auth/login (on success)
- `ml.py` — POST /ml/retrain (on trigger)
- `alerts.py` — PUT /alerts/{id}/status (on status change)
- `reports.py` — POST /reports/generate (on generation)
- `intel.py` — POST /intel/sync (on IOC sync)

Verification:

#	Check	Expected
1	Login → audit entry created	action="login", entity_type="user"
2	Retrain → audit entry	action="model_retrain", entity_type="model"
3	Alert status change → audit entry	action="alert_status_change", entity_type="alert"
4	`GET /admin/audit-log` returns entries	entries[] length > 0
5	Filter by action works	`?action=login` returns only login events

TASK 4 — RBAC Middleware Scaffold ⚪

Time Est: 60 min | **Priority:** ⚪ Medium — Week 6 deliverable
Source: MASTER_DOC_PART5 Week 6 ("RBAC enforcement on all endpoints")

4.1 Create `backend/app/middleware/rbac.py`

Define role-based access control middleware:

```
"""
ThreatMatrix AI — Role-Based Access Control

Per MASTER_DOC_PART5 Week 6:
  "RBAC enforcement on all endpoints — Role-based access verified"

Roles (per MASTER_DOC_PART3 §11):
  - admin: Full access (user management, config, all endpoints)
  - analyst: Read/write alerts, flows, reports, AI analyst, hunt
  - viewer: Read-only access to dashboards and reports
"""

from enum import Enum
from typing import List
from functools import wraps

from fastapi import Depends, HTTPException, status
from app.dependencies import get_current_user


class UserRole(str, Enum):
    ADMIN = "admin"
    ANALYST = "analyst"
    VIEWER = "viewer"


def require_role(allowed_roles: List[UserRole]):
    """Dependency that checks if the current user has one of the allowed roles."""
    async def _check(current_user = Depends(get_current_user)):
```

```
        if current_user.role not in [r.value for r in allowed_roles]:
            raise HTTPException(
                status_code=status.HTTP_403_FORBIDDEN,
                detail=f"Insufficient permissions. Required: {[r.value for r in
allowed_roles]}",
            )
        return current_user
    return _check
```

4.2 Apply to Protected Endpoints

Endpoint Category	Required Role	Notes
`POST /ml/retrain`	admin	Model modification
`GET /admin/audit-log`	admin	Sensitive data
`POST /reports/generate`	admin, analyst	Report creation
`PUT /alerts/{id}/status`	admin, analyst	Alert triage
`POST /capture/start`, `/stop`	admin	Capture control
All GET endpoints	admin, analyst, viewer	Read access

Verification:

#	Check	Expected
1	Admin role → full access	200 on all endpoints
2	Analyst role → retrain blocked	403 on POST /ml/retrain
3	Viewer role → write blocked	403 on PUT /alerts/{id}/status
4	No token → 401	Unauthorized response
5	RBAC decorator reusable	`require_role([UserRole.ADMIN])` pattern

TASK 5 — LLM Budget Tracking Enhancement ⚪

Time Est: 30 min | **Priority:** ⚪ Low — Early Week 6 prep
Source: MASTER_DOC_PART5 Week 6 ("LLM budget tracking + caching")

The `GET /llm/budget` endpoint exists. Enhance with actual token tracking.

5.1 Add Token Tracking to LLM Gateway

Update `llm_gateway.py` to track:

- Total tokens used (input + output) per request
- Running cost estimate based on model pricing
- Request count per model

- Budget remaining (from `LLM_MONTHLY_BUDGET_USD` env var)

5.2 Persist Budget Data

Store in Redis for cross-request aggregation:

- `llm:budget:tokens_used` — total tokens this month
- `llm:budget:requests` — request count per model
- `llm:budget:cost` — estimated cost

Verification:

#	Check	Expected
1	`GET /llm/budget` returns real data	tokens_used > 0, cost > 0
2	Token count increases per request	Tracked in Redis
3	Budget remaining calculated	`monthly_budget - cost_used`

Files Modified / Created (Expected)

File	Action	Lines (est.)
`ml/saved_models/eval_results/cicids2017_validation.json`	CREATE	~50 (output file)
`app/services/report_generator.py`	CREATE	~200
`app/services/audit_service.py`	CREATE	~60
`app/middleware/rbac.py`	CREATE	~50
`app/api/v1/auth.py`	MODIFY	+5 (audit logging)
`app/api/v1/ml.py`	MODIFY	+5 (audit logging)
`app/api/v1/alerts.py`	MODIFY	+5 (audit logging)
`app/api/v1/reports.py`	MODIFY	+30 (PDF generation)
`app/services/llm_gateway.py`	MODIFY	+30 (token tracking)
`requirements.txt`	MODIFY	+1 (reportlab)

Risk Assessment

Risk	Impact	Mitigation
CICIDS2017 dataset too large for VPS	Validation fails	Use single-day CSV subset (~170 MB) or synthetic subset
ReportLab not in Docker image	PDF generation fails	Add to requirements.txt, rebuild

Risk	Impact	Mitigation
Audit log inserts slow endpoint responses	Performance impact	Fire-and-forget with `asyncio.create_task()`
RBAC breaks existing tests/workflows	Access errors	DEV_MODE bypass for development
LLM providers don't report token counts consistently	Budget tracking inaccurate	Estimate from response length

---

Day 17 → Day 18 Bridge

If all 5 tasks complete, Day 18 should focus on:

- Week 6 continuation: User management admin endpoints
- System health monitoring endpoint enhancement
- Network Flow connection graph data endpoint
- Next.js build error resolution (for deployment readiness)

STRICT RULES REMINDER

1. **DO NOT** change ensemble weights (0.30/0.45/0.25) — LOCKED
2. **DO NOT** change alert thresholds (0.90/0.75/0.50/0.30) — LOCKED
3. **DO NOT** suggest Kafka, Kubernetes, Elasticsearch, MongoDB
4. **DO NOT** add features not in the 10 modules
5. **DO NOT** use Tailwind CSS — Vanilla CSS + CSS Variables only
6. LLM via **OpenRouter only** — 3 verified models
7. Master documentation (5 parts) is the **SOLE source of truth**
8. All code: **typed, error-handled, documented, production-quality**
9. Python: **type hints, async/await, SQLAlchemy 2.x**

Day 17 Worklog — Week 5 Day 2
v0.4.0 Critical MVP: ACHIEVED ☑ | v0.5.0 Feature Depth: ~90% (CICIDS2017 remaining)
API Coverage: 46/46 (100%) 🎯
Target: CICIDS2017 validation + PDF reports + audit wiring + RBAC + LLM budget
Ensemble: 80.73% acc | 80.96% F1 | 0.9312 AUC-ROC (LOCKED)
IOC Database: 1,367 indicators (§11.3 fully compliant)